

SN360/SN361 Introduction à la conception des circuits numériques

Module « Systèmes matériels et logiciels »

Crédit SN360 : 4/4 / SN361 : 2/6

Vincent Berouille

Bureau : D202

vincent.berouille@esisar.grenoble-inp.fr

Plan

I Introduction

II Représentation des nombres en binaire

III Circuits combinatoires

IV Circuits séquentiels

V Machine à états finis

VI Compléments HDL

VII Circuits reconfigurables (optionnel)

VI Compléments HDL

- Paramètre générique
- Boucles FOR
- Simulation événementielle

VI Compléments HDL

Paramètre générique

- Les paramètres génériques permettent de décrire des entités flexibles
 - par exemple, un additionneur sur N bits (avec N un entier non nécessairement défini)
- Lors de l'instanciation on indique la valeur de N (paramètre générique) qu'utilisera l'instance
 - avec l'exemple précédent, le nombre de bits sur lequel l'addition se fera

VI Compléments HDL

Paramètre générique

VHDL

```
entity countN is  
  generic(N:integer:=10);  
  port(clk, rst, en : in bit;  
        count : out bit_vector(N-1  
  downto 0));  
End countN;
```

optionnel

```
I1: countN  
generic map(N => 3)  
port map(clk, rst, en, count);
```

Verilog

```
module countN  
#( parameter integer N = 10 )  
(input wire clk, input wire rst,  
 input wire en, output reg [N-1:0]  
  count );  
  
countN #( .N(3) ) I1 ( .clk(clk),  
  .rst(rst), .en(en), .count(count) );
```

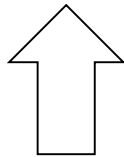
VI Compléments HDL

Boucles **for... loop**

VHDL

```
for i in 10 downto 1 loop  
    -- instructions utilisant i  
    ...  
end loop;
```

Ne pas déclarer la variable i



Seule boucle
utilisée pour la synthèse

Verilog

```
integer i;  
for (i = 10; i >= 1; i = i - 1)  
begin  
    // instructions utilisant i  
    ...  
end
```

Déclarer la variable i

VI Compléments HDL

Exemple d'erreur liée aux boucles **for...loop**

- Que veut-on faire ici?
- Pourquoi cela ne fonctionne pas?
- Que faut-il faire pour corriger cette erreur (sans variable!)?

```
entity mon_xor is  
port(    a_bus : in bit_vector(7 downto 0);  
        x : out bit);  
end mon_xor;  
architecture fonctionne_pas of mon_xor is  
begin  
  p1:process (a_bus)  
    begin  
      x <= a_bus(0) xor a_bus(1);  
      for i in 1 to 6 loop  
        x <= a_bus(i+1) xor x;  
      end loop;  
    end process;  
end fonctionne_pas;
```

VI Compléments HDL

Solution

```
architecture fonctionne of mon_xor is  
signal temp : bit_vector(7 downto 1);  
begin  
  p1:process (a_bus, temp)  
  begin  
    temp(1) <= a_bus(0) xor a_bus(1);  
    for i in 1 to 6 loop  
      temp(i+1) <= a_bus(i+1) xor temp(i);  
    end loop;  
  end process;  
  x <= temp(7);  
end fonctionne;
```


VI Compléments HDL

Boucle **for ... generate**

```
implant : for i in 0 to 3 generate  
    b : bascule  
    port map (h, d(i),  
raz, s(i), sb(i));  
end generate;
```

for... generate permet
de créer
des instances multiples

```
genvar i;  
generate  
    for (i = 0; i < 4; i = i + 1)  
begin : b  
        bascule bascule_inst (  
            .h(h),  
            .d(d[i]),  
            .raz(raz),  
            .s(s[i]),  
            .sb(sb[i])  
        );  
    end  
endgenerate
```

VI Compléments de HDL

Exemple

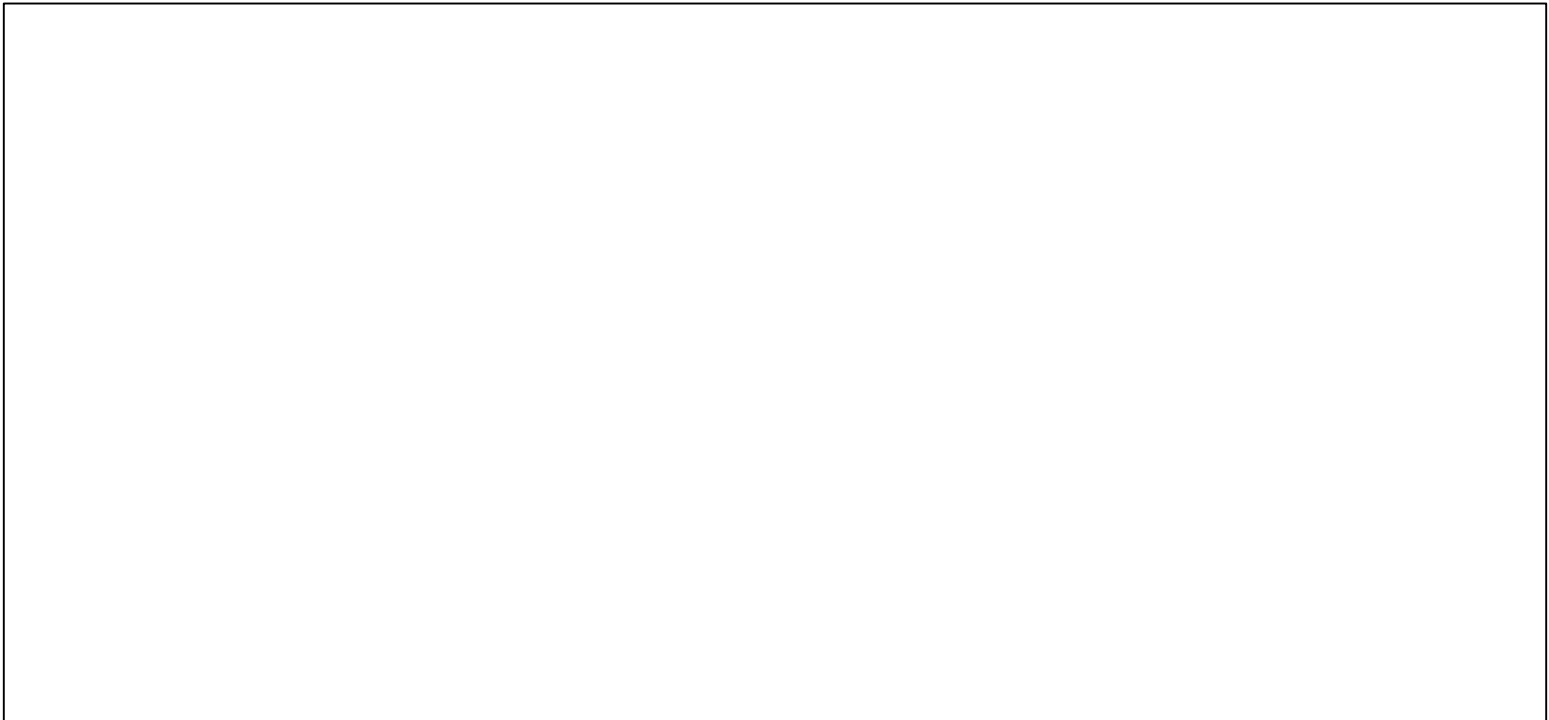
<pre>library IEEE; use IEEE.std_logic_1164.all; entity Nbit_adder is generic(N : integer := 4); port (a,b : in std_logic_vector(N-1 downto 0); ci : in std_logic; sum : out std_logic_vector(N-1 downto 0); co : out std_logic); end entity; architecture arc of Nbit_adder is component FA port(A,B,Cin : in std_logic; S, Cout : out std_logic); end component; signal c :std_logic_vector(N downto 0);</pre>	<pre>begin c(0) <= ci; co <= c(N); G: for i in 0 to N-1 generate inst: FA port map(a(i), b(i), c(i), sum(i), c(i+1)); end generate G; end arc;</pre>
--	---

Additionneur
binaire générique sur N bits

VI Compléments HDL

Exercice SIPO générique en description structurelle

Donnez une description structurelle d'un registre N bits avec une entrée série DIN et une sortie parallèle DOUT sur N bits basé sur des composants FLIPFLOP



VI Compléments HDL

Simulation événementielle

- En VHDL, ils existent plusieurs types d'objets :
 - constante : la valeur portée ne change pas
 - variable : affectation immédiate de sa valeur
 - **signal : NOUVEAU TYPE D'OBJET**

Les signaux permettent de modéliser le parallélisme du matériel :

plusieurs processus peuvent s'exécuter simultanément

VI Compléments HDL

Simulation événementielle

- Contrairement à une variable, dont l'affectation est immédiate

**Un signal n'est pas affecté immédiatement
MAIS uniquement lorsque l'exécution de tous les
processus en cours est suspendue (Δ)**

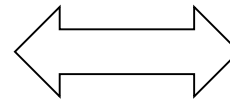
VI Compléments HDL

Simulation événementielle

- Définition du pilote :
 - Liste de couples ***date-valeur*** (date comptée relativement à *l'instant de l'affectation*)
 - Exemple :

S <= '0', '1' after 10 ns, '0' after 25 ns;

Le pilote est un tableau
associé à chaque signal
dans la mémoire



Date	Valeur
0 (Δ)	'0'
10ns	'1'
25ns	'0'

Pilote de S

VI Compléments HDL

Simulation événementielle

```
signal h : bit;           ← Zone de déclaration de
                             l'architecture
begin

    horloge : process -- déclaration de processus

-- zone de déclaration du process us

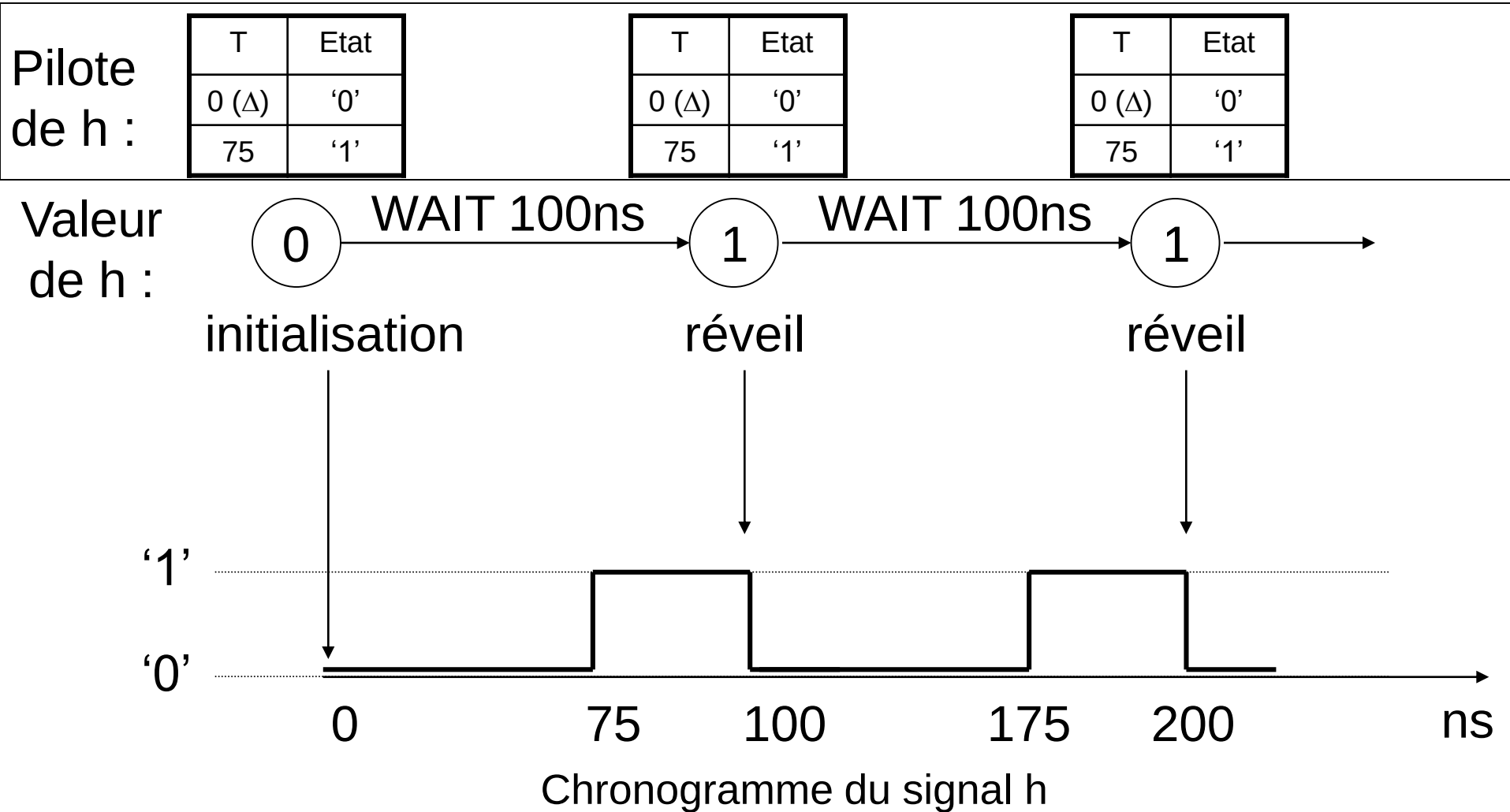
    begin -- début de la zone de définition du processus

        h <= '0', '1' after 75 ns;
        wait for 100 ns;      ← processus suspendu
                               pour une durée
                               déterminée

    end process; -- fin du processus
```

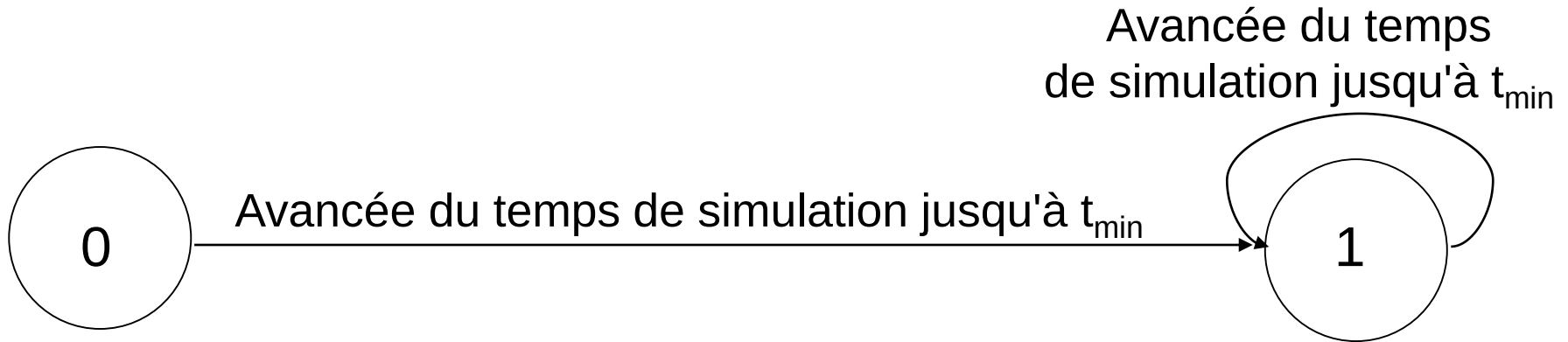
VI Compléments HDL

Simulation événementielle



VI Compléments HDL

Simulation événementielle



Initialisation de tous les objets à leur valeur initiale, puis exécution de tous les processus

Réveil et simulation des processus concernés
Production de nouveaux événements
 $\Rightarrow$ nouveau $t_{\min}$

Si $t_{\min}$ est égal au temps t_0
alors il y a itération à t_0 (ou $t_0 + \Delta$)

VI Compléments HDL

Simulation événementielle

- Instruction **WAIT**

- Synchronise (en les suspendant/réveillant) les processus

- Plusieurs formes :

Non synthétisable!

- **WAIT FOR** *durée*;

- **WAIT ON** *liste de signaux*;

- ...

VI Compléments HDL

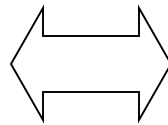
Simulation événementielle

- Equivalence entre *liste de sensibilité* et instruction **WAIT ON**

Liste de sensibilité



```
proc1: process (a, b, c)
begin
  x<=a and b and c;
end process;
```



```
proc2: process
begin
  x<=a and b and c;
  wait on a, b, c;
end process;
```

Pour être synthétisable, tous les signaux lus dans le processus doivent être présents dans la liste de sensibilité (sauf exception)

VI Compléments HDL

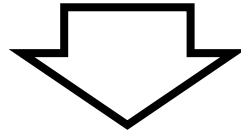
Simulation événementielle

**L'ordre des processus dans un programme n'a pas d'importance,
l'ordre de leurs exécutions dépend des événements sur les signaux de
leur liste de sensibilité**

VI Compléments HDL

Simulation événementielle

Plusieurs composants ne pilotent pas
le même signal

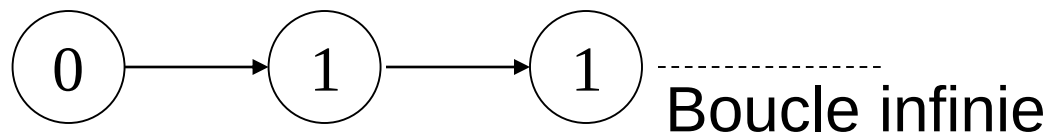


En VHDL, un signal doit être affecté à l'intérieur d'un seul processus

VI Compléments HDL

Simulation événementielle

- Remarque :
 - Il est syntaxiquement possible d'avoir un process sans *liste de sensibilité* ni " **wait** »,
 - Cependant, **le temps de simulation n'avance pas** car ce process reboucle indéfiniment sur lui même (n'est jamais suspendu)



VI Compléments HDL

Simulation événementielle

- En Verilog, en dehors des blocs procéduraux :

assign A = B;

A et B sont affectés en parallèle donc équivalent à A=B=C (un seul fil)

assign B = C;

- Dans les blocs procéduraux « always » :

```
module BlockingEx4
(output reg F,
 input wire A,
 input wire Clock);

reg B;

always @ (posedge Clock)
begin
    B = A;    // statement 1
    F = B;    // statement 2
end

endmodule
```

Affectations immédiates (1 FF)

```
module NonBlockingEx4
(output reg F,
 input wire A,
 input wire Clock);

reg B;

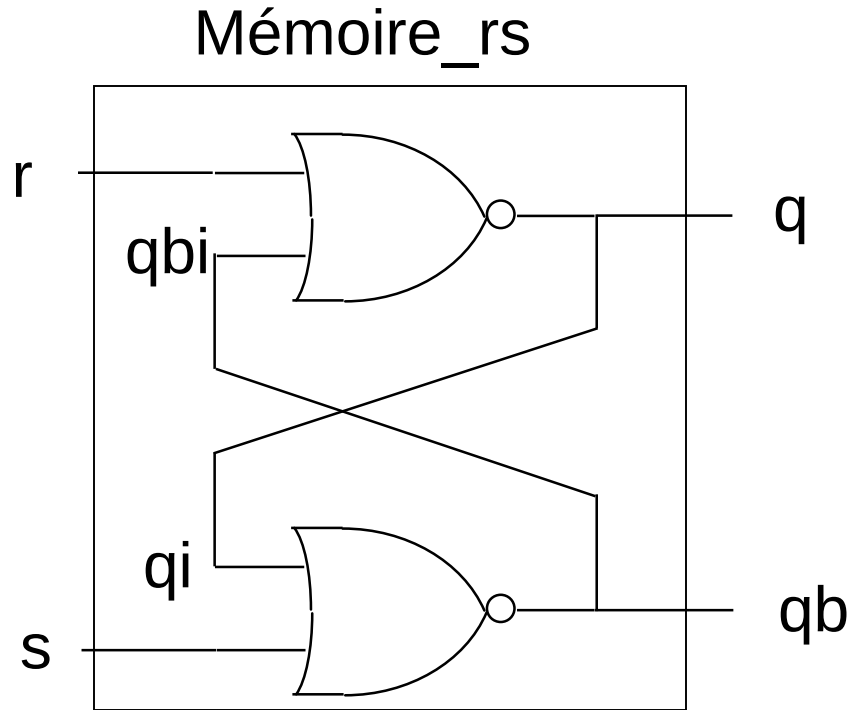
always @ (posedge Clock)
begin
    B <= A;    // statement 1
    F <= B;    // statement 2
end

endmodule
```

Affectations non immédiates (2 FF) 23

VI Compléments HDL

Simulation événementielle



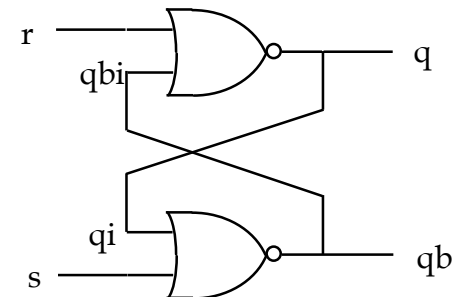
Bascule RS

VI Compléments HDL

Simulation événementielle

Table de vérité de la mémoire RS :

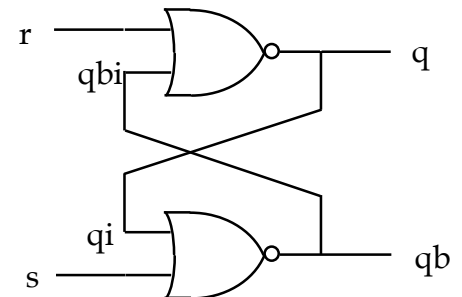
s	r	q	qb	
1	0	1	0	Mise à "1"
0	1	0	1	Mise à "0"
0	0	1	0	Sortie inchangée (après sr = 10)
0	0	0	1	Sortie inchangée (après sr = 01)
1	1	?	?	À ne pas faire!



VI Compléments HDL

Simulation événementielle

```
1 entity memoire_rs is
2     port ( s, r : in bit;
3           q, qb : out bit);
4 end;
5
6 architecture processus of memoire_rs is
7
8     constant tplh : time := 2 ns;
9     constant tphl : time := 1 ns;
10    signal qi : bit := '0';
11    signal qbi : bit := '1';
12 begin
13
```



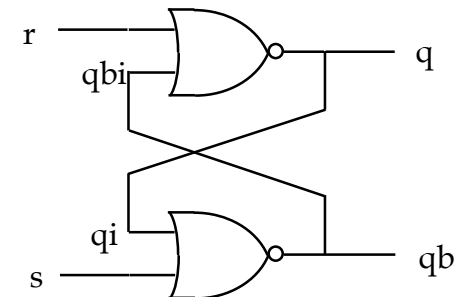
VI Compléments HDL

Simulation événementielle

```
14 n1 : process
15     variable qtmp : bit;
16     begin
17         wait on s, qi ;
18         qtmp := s nor qi; -- là primitive nor
19         if qtmp /= qbi then -- test du changement ?
20             if qtmp = '0' then
21                 qbi <= qtmp after tphl;
22             else
23                 qbi <= qtmp after tplh;
24             end if;
25         end if;
26     end process;
```

déclaration

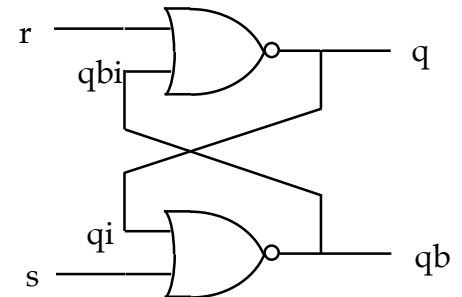
commentaires



VI Compléments HDL

Simulation événementielle

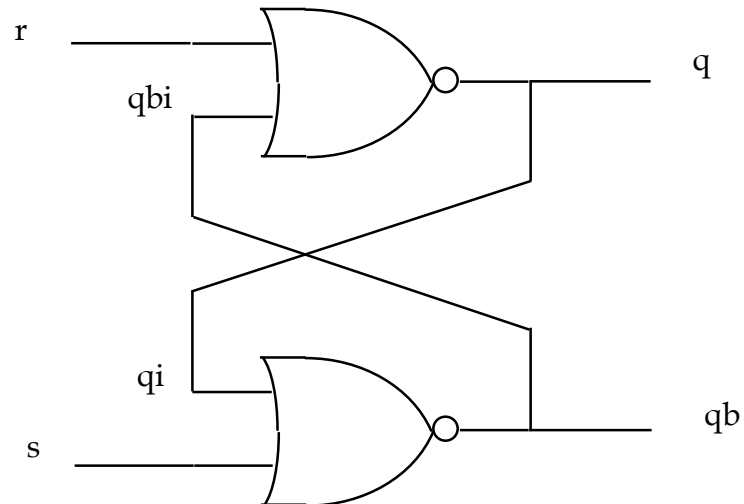
```
28 n2 : process
29     variable qtmp : bit;
30     begin
31         wait on r, qbi ;
32         qtmp := r nor qbi; -- la primitive nor
33         if qtmp /= qi then -- test du changement?
34             if qtmp = '0' then
35                 qi <= qtmp after tphl;
36             else
37                 qi <= qtmp after tplh;
38             end if;
39         end if;
40     end process;
```



VI Compléments HDL

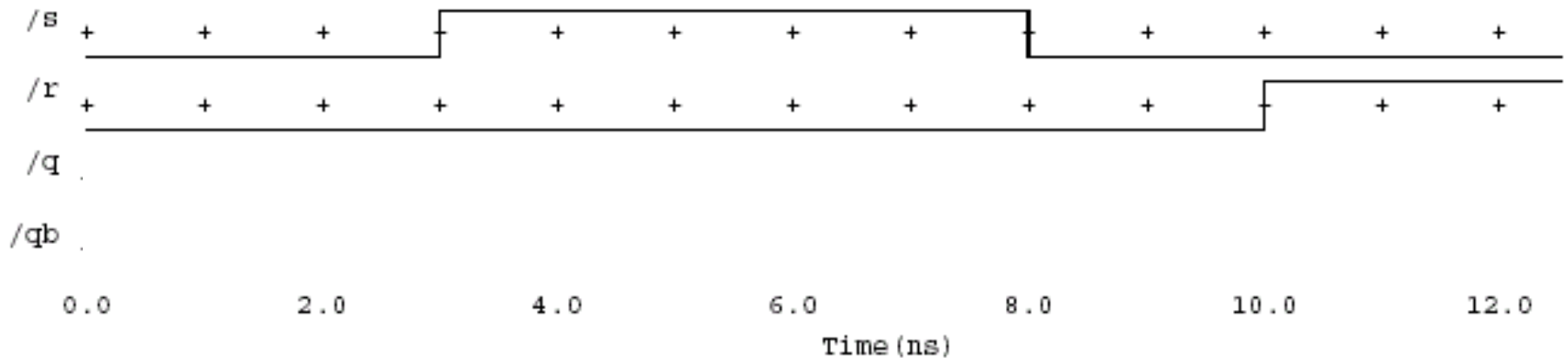
Simulation événementielle

```
41 q <= qi;  
42 qb <= qbi;  
43 end;
```



VI Compléments HDL

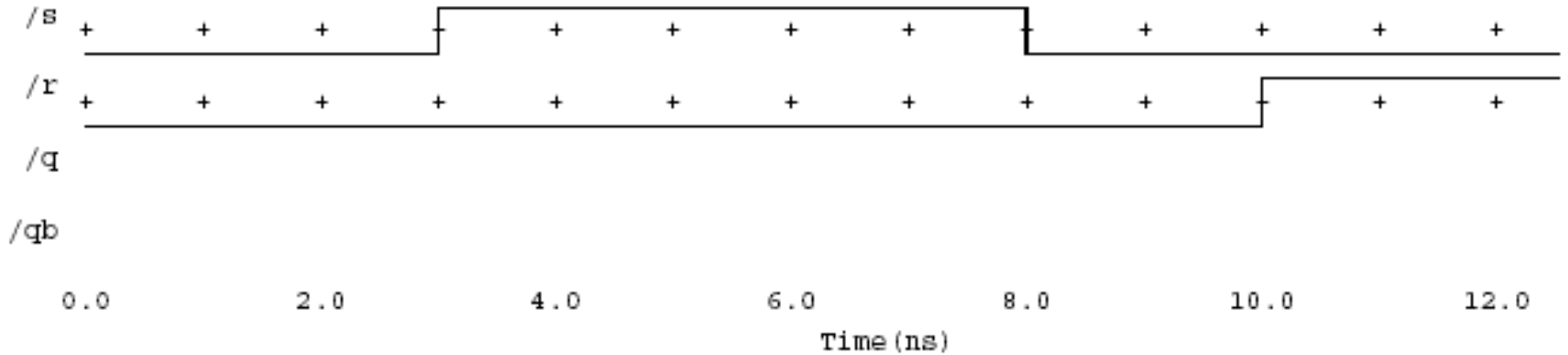
Simulation événementielle



Chronogramme de la bascule RS à compléter

VI Compléments HDL

Simulation événementielle



A continuer...

Faire le QCM : QCM6 SN360/SN361 Compléments
HDL